

From foundations to reliable delivery.

A presentation of what I built, what the evidence shows, and what I learned.

PRESENTED BY

Mark Andrei Castillo

Linux | Docker | CI/CD | Kubernetes HA

THE STORY

Foundation
Packaging
Automation
Availability
Reflection



OVERALL APPROACH

I worked from the foundation upward

01 | FOUNDATION

Linux commands
Permissions
Logs and processes
Bash scripts

02 | PACKAGING

FastAPI API
Next.js UI
MySQL dependency
Docker Compose

03 | DELIVERY

GitHub Actions
Build and test
Docker Hub tags
Notifications

04 | RESILIENCE

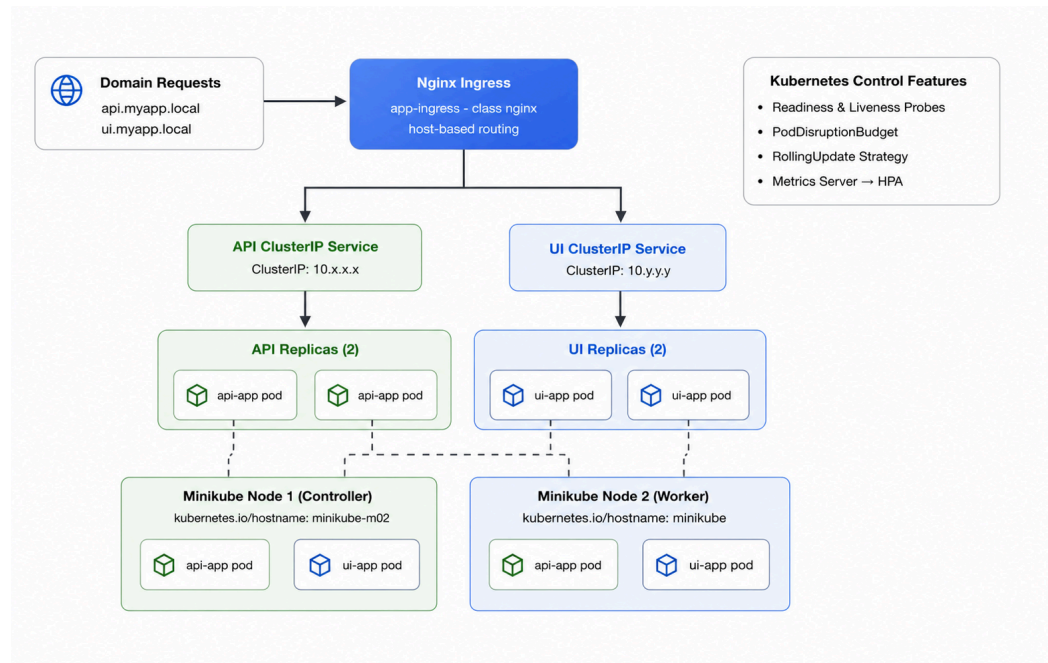
Kubernetes
Two nodes
Replicas and probes
Failover evidence

Part 5 is the explanation: approach, architecture, decisions, challenges, trade-offs, and improvements.

ARCHITECTURE

One application path, from local development to HA runtime

APPLICATION AND DEPLOYMENT ARCHITECTURE



REPOSITORY ARCHITECTURE EVIDENCE

LOCAL

Compose connects the API, UI, and MySQL for repeatable development.

DELIVERY

GitHub Actions verifies the change before publishing traceable container tags.

RUNTIME

Kubernetes adds replicas, Services, Ingress, health checks, scaling, and disruption protection.

I made the machine state visible before building on top of it

SYSTEM HEALTH AND OPERATIONAL CHECKS

```
drainm@aloof:~$ nano devops-exam/scripts/system_health.sh
drainm@aloof:~$ chmod +x devops-exam/scripts/system_health.sh
bash devops-exam/scripts/system_health.sh

=====
SYSTEM HEALTH REPORT
=====

[1] DATE & TIME
Thu Aug 6 11:52:56 CST 2026

[2] UPTIME
11:52:56 up 14 min,  1 user,  load average: 0.02, 0.23, 0.15

[3] MEMORY USAGE
      total      used      free   shared buff/cache   available
Mem:   1.9Gi  584Mi   96Mi    3.1Mi    1.4Gi    1.4Gi
Swap:   0B       0B       0B

[4] DISK SPACE
Filesystem      Size  Used Avail Use% Mounted on
none            987M   0 987M   0% /usr/lib/modules/5.15.167.4-microsoft-standard-WSL2
none            987M   4.0K 987M   1% /mnt/wsl
drivers          465G   390G   75G   84% /usr/lib/wsl/drivers
/dev/sdb         1007G   2.2G   994G   1% /
none            987M   80K 987M   1% /mnt/wslg
none            987M   0 987M   0% /usr/lib/wsl/lib
rootfs           987M   2.4M 987M   1% /init
none            987M   496K 986M   1% /run
none            987M   0 987M   0% /run/lock
none            987M   0 987M   0% /run/shm
tmpfs            4.0M   0 4.0M   0% /sys/fs/cgroup
none            987M   76K 987M   1% /mnt/wslg/versions.txt
none            987M   76K 987M   1% /mnt/wslg/doc
C:\             465G   390G   75G   84% /mnt/c
D:\             50M   16M   35M   32% /mnt/d
H:\            932G   845G   87G   91% /mnt/h
tmpfs           100M   16K 100M   1% /run/user/1000

[5] TOP 5 PROCESSES BY CPU
USER      PID %CPU %MEM    VSZ   RSS TTY      STAT START   TIME COMMAND
root         53  0.0  0.7 66828 16092 ?        Ss   11:38   0:05 /usr/lib/systemd/systemd-journald
root         1  0.2  0.6 22108 13464 ?        Ss   11:38   0:02 /sbin/init
root      1156  0.1  3.8 2034728 61048 ?        Ssl  11:42   0:01 /usr/bin/containerd
root      1207  0.0  5.0 2130632 101780 ?        Ssl  11:42   0:00 /usr/bin/dockerd -H fd:// --containerd=/run/containerd/containerd
.sock
message+  100  0.0  0.2 9772 5300 ?        Ss   11:38   0:00 @dbus-daemon --system --address=systemd: --nofork --nopidfile --s
systemd-activation --syslog-only

[6] CPU INFO
#
0.02 0.23 0.15 1/191 2795

=====
END OF REPORT
=====
drainm@aloof:~$
```

LOCAL WSL CAPTURE

WHAT I COMPLETED

File operations, permissions, searching, processes, networking, packages, users, archives, and Bash scripts.

WHAT I LEARNED

Repeatable checks are more useful than isolated commands. The same habit helped later with Docker and Kubernetes troubleshooting.

API

UI

WHY THIS DESIGN

Separate runtimes make the services easier to build, test, and scale independently.

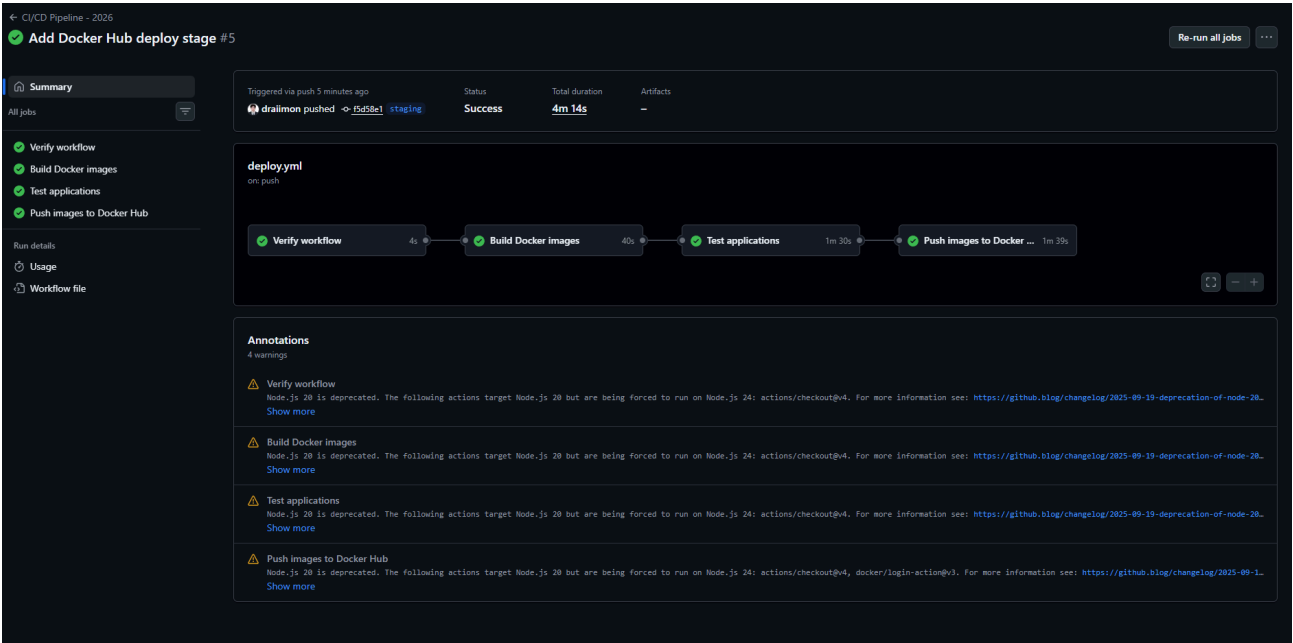
[illegible]

LOCAL WSL CAPTURE

Result: the API, UI, and MySQL services were running and healthy before automation.

The pipeline is a quality gate, not only a deployment button

SUCCESSFUL GITHUB ACTIONS PIPELINE



CHECKPOINTED CI EVIDENCE

TRIGGER

Push to staging starts the workflow.

QUALITY

Verify, build, and test run before publishing.

TRACEABILITY

SHA, staging, and latest tags identify the result.

RESULT

Build, test, publish, notify: passed.

The captured result shows the delivery path completed

SUCCESS NOTIFICATION EVIDENCE



[draimon/draimon-devops-playground] Run succeeded: CI/CD Pipeline - 2026, Attempt #2 - staging (f5d58e1) [View](#)

 **Mark Andrei Castillo** [monfactions@gmail.com](#)
to draimon/draimon-devops-playground, CI •

 [draimon/draimon-devops-playground] CI/CD Pipeline - 2026 workflow run, Attempt #2



CI/CD Pipeline - 2026, Attempt #2: All jobs were successful

[View workflow run](#)

Status	Job	Annotations
✓	CI/CD Pipeline - 2026, Attempt #2 / Verify workflow Succeeded in 4 seconds	1
✓	CI/CD Pipeline - 2026, Attempt #2 / Build Docker images Succeeded in 39 seconds	1
✓	CI/CD Pipeline - 2026, Attempt #2 / Test applications Succeeded in 1 minute and 45 seconds	1
✓	CI/CD Pipeline - 2026, Attempt #2 / Push Images to Docker Hub Succeeded in 1 minute and 38 seconds	1

CHECKPOINTED EVIDENCE

LATER RUNTIME EVIDENCE

[illegible]

PART 4 RUNTIME EVIDENCE

HONEST EVIDENCE BOUNDARY

I chose Kubernetes because the controls match the requirement

REDUNDANCY

- Two API replicas
- Two UI replicas
- Two Minikube nodes
- Topology spread
- RollingUpdate

TRAFFIC

- ClusterIP Services
- Ingress host routing
- Ready endpoints
- Local domain access
- Stable service path

RECOVERY

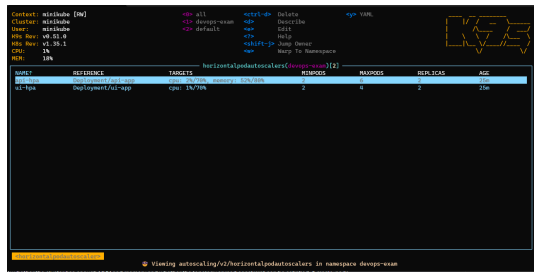
- Readiness and liveness
- HPA scaling
- PodDisruptionBudget
- Node-failure test
- Automatic replacement

Trade-off: Minikube is practical and reproducible for the exam, but a managed cluster would be the production next step.

PART 4 | RUNTIME EVIDENCE

The runtime checks the story the manifests promised

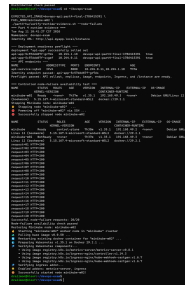
HPA METRICS AND REPLICA STATE



```
Context: minikube [m]
Cluster: minikube
User: minikube
API Ver: v1.20.0
K8s Ver: v1.20.0
CPU: 1%
MEM: 10%
HorizontalPodAutoscaler: 1/1
NAME REFERENCE TARGETS CURRENT DESIRED MINPODS MAXPODS ELAPSED AGE
-----
horizontalpodautoscaler/... 2 0 2 21m
```

LOCAL WSL / K9S CAPTURE

NODE FAILURE: 20/20 HTTP 200



```
NAME STATUS AGE CPU MEMORY DISK NETWORK PODS
-----
node/... 1m 10% 10% 10% 10% 10% 10%
```

LOCAL WSL CAPTURE

WHAT THE EVIDENCE SHOWS

The deployment was not treated as complete because the YAML existed.

The captured checks show:

- ready replicas and expected images
- Service endpoint membership
- domain-based routing
- traffic reaching both replicas
- HTTP 200 responses while one node was unavailable

CHALLENGES AND DECISIONS

The hardest problems were boundary problems

SOURCE AND IMAGE

Challenge: early containers did not contain the real application source.
Resolution: clone the source and rebuild from the correct context.

DATABASE ALIGNMENT

Challenge: the initial database setup did not match the API.
Resolution: align the Compose stack with MySQL.

FRESH CI RUNNERS

Challenge: one job cannot rely on state from another job.
Resolution: each job declares its own setup and readiness checks.

RUNTIME TRUTH

Challenge: rollout success does not guarantee traffic health.
Resolution: preflight images, endpoints, routing, and real requests.

Decision principle: make the boundary observable before trusting the next layer.

I can explain what is complete and what I would improve

WHY THESE TOOLS

Docker Compose: simple local feedback loop.

GitHub Actions: repeatable checks on a fresh runner.

Kubernetes: replicas, routing, health, scaling, and recovery in one model.

Minikube: realistic Kubernetes behavior without cloud setup.

WITH MORE TIME

Add vulnerability scanning as a required policy gate.

Deploy with immutable image digests.

Run and document a real rollback test.

Move from local Minikube to managed staging.

Add metrics, logs, and alerting beyond screenshots.

Reliable delivery is a chain of verified boundaries

Linux made the system observable.

Docker made services portable.

CI/CD made delivery repeatable.

Kubernetes made availability testable.

Questions and discussion.
